Pattern Recognition 42 (2009) 867 – 870

Contents lists available at ScienceDirect

Pattern Recognition

journal homepage: www.elsevier.com/locate/pr

Improvement of the fast exact pairwise-nearest-neighbor algorithm

Yi-Ching Liaw

Department of Computer Science and Information Engineering, Nanhua University, Chiayi 622, Taiwan, ROC

ARTICLE INFO

Article history:
Received 27 March 2008
Received in revised form 25 August 2008
Accepted 2 October 2008

Keywords:
Data clustering
Pairwise-nearest-neighbor
Fast search algorithm

ABSTRACT

Pairwise-nearest-neighbor (PNN) is an effective method of data clustering, which can always generate good clustering results, but with high computational complexity. Fast exact PNN (FPNN) algorithm proposed by Fränti et al. is an effective method to speed up PNN and generates the same clustering results as those generated by PNN. In this paper, we present a novel method to improve the FPNN algorithm. Our algorithm uses the property that the cluster distance increases as the cluster merge process proceeds and adopts a fast search algorithm to reject impossible candidate clusters. Experimental results show that our proposed method can effectively reduce the number of distance calculations and computation time of FPNN algorithm. Compared with FPNN, our proposed approach can reduce the computation time and number of distance calculations by a factor of 24.8 and 146.4, respectively, for the data set from three real images. It is noted that our method generates the same clustering results as those produced by PNN and FPNN.

1. Introduction

Data clustering is usually used to classify data points into several clusters. There are many applications for data clustering including vector quantization (VQ) [1–3], pattern recognition [4], knowledge discovery [5], speaker recognition [6], fault detection [7], and web mining [8].

To classify data points into clusters is not an easy task and usually requires a large amount of computations. Many heuristic methods are proposed to solve this problem [9–11]. Among the available approaches, pairwise-nearest-neighbor (PNN) algorithm [11] can usually generate a better clustering result [12]. However, for a data set of N points, the computational complexity of PNN algorithm is $O(N^3)$. This algorithm is impractical for a large data set.

To reduce the complexity of PNN, Kurita [13] proposed a method to store the distances of all pairs of clusters into a heap structure. This method requires additional memory by an order of N^2 and can reduce the computational complexity to $O(N^2 \log_2 N)$. Fränti et al. [14] presented another fast PNN algorithm (referred to as FPNN). FPNN requires only $O(N)$ memory and has a complexity of $O(\tau N^2)$, where τ is about four [14].

FPNN spends most computational time on finding the nearest neighbor for a cluster. In this paper, we will present a new method to speed up the process of determining the nearest neighbor for a cluster. It is shown in Ref. [15] that the distance between two clusters increases after the process of merging two clusters of the nearest pair. We use this property to reject clusters, which are impossible to be the nearest neighbor of a cluster. Secondly, a fast search algorithm proposed by Lai and Liaw [16] is modified and applied to speed up the process of determining the nearest neighbor for each cluster. Through the utilization of these two techniques, our proposed method can effectively reduce the computation time and number of distance calculations of FPNN. Compared with FPNN, our method requires $O(N)$ additional memory to store more information for every cluster.

This paper is organized as follows. Section 2 describes the fast exact PNN algorithm proposed by Fränti et al. Section 3 presents the algorithm developed in this paper. Some experimental results are given in Section 4 and concluding remarks are presented in Section 5.

2. Fast exact PNN algorithm

Given a set of N data points, the PNN algorithm [11] is used to cluster data points into M clusters through a series of merging processes. In the beginning, the number of clusters is set to N and each cluster contains only one data point. The merging process of PNN includes two steps: finding the nearest pair of clusters from all clusters and merging this nearest pair into a new cluster. After a merging process, the number of clusters will be decreased by one. The merging process is repeated until the number of clusters is equal to M.

E-mail address: ycliaw@mail.nhu.edu.tw

0031-3203/$ - see front matter © 2008 Elsevier Ltd. All rights reserved.
doi:10.1016/j.patcog.2008.10.001

To find the nearest pair of clusters, the distances for all pairs of clusters should be calculated. Let R_a and R_b be two clusters, $\mathbf{C}_a$ be the center of R_a, and $\mathbf{C}_b$ be the center of R_b. Let n_a be the number of data points in R_a and n_b be the number of data points in R_b. The distance between two clusters is defined as follows.

$$D_{a,b} = \frac{n_a n_b}{n_a + n_b} d(\mathbf{C}_a, \mathbf{C}_b) \tag{1}$$

where $d(\mathbf{C}_a,\mathbf{C}_b)$ is the squared Euclidean distance between $\mathbf{C}_a$ and $\mathbf{C}_b$. Let $\mathbf{X} = [x_1, x_2, \ldots, x_d]^t$ and $\mathbf{Y} = [y_1, y_2, \ldots, y_d]^t$. The squared Euclidean distance $d(\mathbf{X},\mathbf{Y})$ is defined as

$$d(\mathbf{X}, \mathbf{Y}) = \sum_{k=1}^{d} |x_k - y_k|^2 \tag{2}$$

Let R_{ab} be the new cluster obtained through merging R_a and R_b. The cluster center $\mathbf{C}_{ab}$ of R_{ab} and the number of data points in R_{ab} are calculated by the following equations:

$$\mathbf{C}_{ab} = (n_a \mathbf{C}_a + n_b \mathbf{C}_b)/(n_a + n_b) \tag{3}$$

$$n_{ab} = n_a + n_b \tag{4}$$

After the merging process, cluster R_a is replaced by R_{ab} and cluster R_b should be deleted.

The computational complexity of PNN is $O(N^3)$, which is come from the process of finding the nearest pair of clusters. To reduce the complexity of PNN, Fränti et al. [14] proposed a fast exact PNN method (referred to as FPNN). This method used a nearest neighbor table *NN* and a nearest distance table *ND*, which, respectively, records the nearest neighbor and the distance to the nearest neighbor for every cluster to reduce the computational complexity. It is noted that $NN[i]$ and $ND[i]$, respectively denotes the nearest neighbor and distance to the nearest neighbor of cluster R_i. If two clusters R_a and R_b are merged, then tables *NN* and *ND* must be updated. Since the distance to the nearest neighbor of a cluster increases monotonically as the merging process proceeds, only items of tables corresponding to clusters with either cluster R_a or cluster R_b as their nearest neighbor should be updated. Let $S = \{R_i : NN[i] = R_a \text{ or } NN[i] = R_b\} \backslash \{R_a, R_b\}$ be the set of clusters whose items in tables *NN* and *ND* should be updated. Since the complexity of finding the nearest neighbor for a cluster is $O(N)$, the complexity of updating tables *NN* and *ND* is $O(\tau N)$, where τ is the number of clusters in set *S*. Thus, the computational complexity of FPNN is $O(\tau N^2)$. Compared with PNN, FPNN requires $O(N)$ additional memory to store the nearest neighbor and the distance to the nearest neighbor for every cluster.

3. Proposed algorithm

In this section, we will present our modified FPNN algorithm (referred to as MFPNN) and MFPNN using fast search.

3.1. Modified FPNN algorithm

After two clusters are merged, the nearest neighbor of a cluster may need to be updated. Let r be the distance between a cluster R_i and its candidate nearest neighbor R_t. If

$$ND[j] \geqslant r \tag{5}$$

then R_j is impossible to be the nearest neighbor of R_i, since $ND[j]$ is the nearest distance between R_j to its nearest neighbor. Using this property, we may present our modified FPNN (MFPNN) algorithm as follows:

Modified FPNN Algorithm

Step 1: Set $N_c = N$ and $ND[i] = 0$ $(i = 1, 2, \ldots, N)$.

Step 2: For a cluster R_i $(i = 1, 2, \ldots, N)$,

Step 2.1: Set $nn =$ -1 and $r = \infty$.

Step 2.2: For each cluster R_j $(j = 1, 2, \ldots, N$ and $j \neq i)$, if $(ND[j] \geqslant r)$, reject cluster R_j. Otherwise, compute $D_{i,j}$. If $(D_{i,j} < r)$, set $r = D_{i,j}$ and $nn = j$.

Step 2.3: Set $NN[i] = nn$ and $ND[i] = r$.

Step 3: Find the cluster pair (R_a, R_b), which has the minimum distance.

Step 4: Merge clusters R_a and R_b into cluster R_{ab}. Update $\mathbf{C}_{ab}$ and n_{ab}. Set $n_a = n_{ab}$, $\mathbf{C}_a = \mathbf{C}_{ab}$ and delete cluster R_a.

Step 5: Find the set $S = \{R_i : NN[i] = R_a \text{ or } NN[i] = R_b\} \backslash \{R_a, R_b\}$.

Step 6: For each cluster R_i in *S*, use the similar procedures described in steps 2.1–2.3 to update $NN[i]$ and $ND[i]$.

Step 7: Set $N_c = N_c - 1$. If $N_c > M$, go to step 3.

Compared to FPNN, there is no additional memory required by MFPNN. That is, the memory complexity of MFPNN is the same as that of FPNN. Let p be the possibility of a cluster cannot be rejected by Eq. (5). The computational complexity of MFPNN will be $O(p\tau N^2)$. In our experiments, the value of p is about to 88.2%. Thus, the computational complexity of MFPNN is about to $O(0.882 \times \tau N^2)$.

3.2. MFPNN using fast search

There are many fast search algorithms available [16–19] to generate the same result as that produced by full search. Here, a fast search algorithm proposed by Lai and Liaw [16] (referred to as Lai's method) is chosen in this paper for its simplicity and good efficiency.

Given a cluster R_q of dimension d with center $C_q = [q_1, q_2, \ldots, q_d]^t$, our problem is to find the nearest cluster of R_q from a given cluster set $S = \{R_i | \ i = 1, 2, \ldots, N\}$. Let n_i be the number of points in cluster R_i and $C_i = [c_{i1}, c_{i2}, \ldots, c_{id}]^t$ be the corresponding cluster center. The distance between clusters R_q and R_i is defined by Eq. (1).

To speed up the process of determining the nearest cluster for cluster R_q, four inequalities are used to check whether a cluster R_i $(i = 1, 2, \ldots, N)$ is impossible to be the nearest neighbor of R_q. Let $\mathbf{v}_1$, $\mathbf{v}_2$ and $\mathbf{v}_3$ be three eigenvectors corresponding to three largest eigenvalues for a data set. The reason of using three projection values is that it has least computing time in the process of nearest neighbor search [20]. Denote the axis in the direction of $\mathbf{v}_k$ $(k = 1, 2, 3)$ as the kth axis. Let q'_k be the projection value of C_q on the kth axis. Similarly denote c'_{ik} as the projection value of C_i on the kth axis. To speed up the searching process, all clusters are sorted in the ascending order of the projections of their centers to the first axis. The searching process is started from a cluster R_{init}, whose center projection on the first axis is closest to q'_1, and followed by R_{init-1}, R_{init+1}, R_{init-2}, R_{init+2}, ..., etc. In the process of searching the nearest cluster for a query cluster R_q, inequalities (6a–c) are first used to reject unlikely clusters.

$$(q'_1 - c'_{i1})^2 \times 0.5 \geqslant d_{min} \tag{6a}$$

$$(q'_2 - c'_{i2})^2 \times \frac{n_q n_i}{n_q + n_i} \geqslant d_{min} \tag{6b}$$

$$(q'_3 - c'_{i3})^2 \times \frac{n_q n_i}{n_q + n_i} \geqslant d_{min} \tag{6c}$$

where 0.5 in inequality (6a) is the minimum value of $n_q n_i/n_q + ni$ for $n_q = n_i = 1$ and d_{min} is the minimum distance between the query cluster R_q and visited clusters. For the case that a cluster R_i is in the front of R_{init} (that is $i < init$) and the inequality (6a) is satisfied, all clusters from R_1 to R_i may be deleted for they are unlikely to be the nearest neighbor of R_q. If a data point R_i is behind the R_{init} (that is $i > init$) and the inequality (6a) is satisfied, all clusters from R_i to R_N should be removed also. Once a cluster cannot be eliminated by inequality (6a), inequalities (6b) and (6c) may be used to reject this cluster.

If a cluster cannot be deleted by inequalities (6a) to (6c), the following inequality is further used to reject the cluster.

$$([(q'_1 - c'_{i1})^2 + (q'_2 - c'_{i2})^2 + (q'_3 - c'_{i3})^2] + (|\mathbf{s_{q_i}}| - |\mathbf{s_{c_i}}|)^2) \times \frac{n_q n_i}{n_q + n_i} \geqslant d_{min} \quad (7)$$

where $\mathbf{s_q}$ and $\mathbf{s_{c_i}}$ are defined as follows.

$$|\mathbf{s_q}| = (|Cq|^2 - |q'_1\mathbf{v}_1 + q'_2\mathbf{v}_2 + q'_3\mathbf{v}_3|^2)^{0.5} \quad (8a)$$

$$|\mathbf{s_{c_i}}| = (|C_i|^2 - |c'_{i1}\mathbf{v}_1 + c'_{i2}\mathbf{v}_2 + c'_{i3}\mathbf{v}_3|^2)^{0.5} \quad (8b)$$

In the case that both inequalities (6a–c) and (7) are unsatisfied, the distance between clusters R_q and R_i will be computed and d_{min} is updated.

As mentioned above, to apply the fast search algorithm for MFPNN, the clusters should be sorted in the ascending order of projections of cluster centers on the first axis. To keep the set of clusters sorted in the ascending order of projections of cluster centers on the first axis, the set of clusters S is sorted first. After a cluster merging process, the order of clusters in S must be updated. To update clusters in S, R_a and R_b are deleted from S and the projection of the cluster center of R_{ab} on the first axis is calculated. Then, we should insert R_{ab} into S. This insertion process can be done in $\log_2 N$ steps using binary search. The detailed procedure of modified FPNN algorithm using fast search (referred to as MFPNN_FS) is presented in the following.

MFPNN using fast search

Step 1: Determine the projection values c'_{ik} and $|\mathbf{s_{c_i}}|$ ($i = 1, 2, \ldots, N$ and $k = 1, 2, 3$) for all cluster centers, where $|\mathbf{s_{c_i}}|$can be calculated using Eq. (8a) or (8b). Arrange clusters in the ascending order of the projection values of cluster centers on the first axis. Set $N_c = N$ and $ND[i] = 0$ ($i = 1, 2, \ldots, N$).

Step 2: For each cluster R_i ($i = 1, 2, \ldots, N$),

Step 2.1: Set $nn = -1$ and $r = \infty$.

Step 2.2: For each cluster R_j ($j = 1, 2, ..., N$ and $j \neq i$), if ($ND[j] \geqslant r$), reject cluster R_j. Otherwise, using inequalities (6a–c) and (7) to reject cluster R_j. If cluster R_j cannot be rejected by inequalities (6a–c) and (7), compute $D_{i,j}$. If ($D_{i,j} < r$), set $r = D_{i,j}$ and $nn = j$.

Step 2.3: Set $NN[i] = nn$ and $ND[i] = r$.

Step 3: Find the pair of clusters (R_a, R_b), which has the minimum distance.

Step 4: Merge clusters R_a and R_b into cluster R_{ab}. Update $\mathbf{C}_{ab}$ and n_{ab}. Determine the projection values $c'_{(ab)k}$(k = 1,2,3) and $|\mathbf{s_{c_{ab}}}|$ for cluster R_{ab}. Set $n_a = n_{ab}$, $\mathbf{C}_a = \mathbf{C}_{ab}$ and delete cluster R_b.

Step 5: Update the set of clusters.

Step 6: Find the set $S = \{R_i : NN[i] = R_a \text{ or } NN[i] = R_b\} \backslash \{R_a, R_b\}$.

Step 7: For each cluster R_i in S, use the similar procedures described in steps 2.1–2.3 to update $NN[i]$ and $ND[i]$.

Step 8: Set $N_c = N_c - 1$. If $N_c > M$, go to step 3.

The memory complexity of MFPNN_FS is O(N), since it requires memory to store the nearest distance, nearest neighbor, and projection values for every cluster. Compared with FPNN and MFPNN, MFPNN_FS requires O(N) additional memory to save the projection values of clusters. Let p_2 represent the possibility that a cluster cannot be rejected through using Eqs. (5), (6a–c), and (7). The computational complexity of MFPNN_FS is O($p_2 \tau N^2$). In our experiments, p_2 is less than 2.5% in most cases. That is, the computational complexity of MFPNN_FS is less than O($0.025\tau N^2$) in most cases. However, p_2 is highly influenced by the degree of cluster separation [21]. In the worst case, p_2 will be up to 55%. In such case, the computational complexity of MFPNN_FS is about O($0.55\tau N^2$).

4. Experimental results

To evaluate the performance of the proposed algorithm, several sets of synthetic data and a set of real images have been used. The synthetic data are generated using the method described in Ref. [21]. The proposed methods: MFPNN and MFPNN_FS are compared to PNN [11] and FPNN [14], in terms of the number of distance calculations and computing time. All computing is performed on an Intel Core 2 Duo 2.4 GHz PC with memory of 1 GB. All programs are implemented as console applications of Microsoft Visual C++ 6.0 and are executed under Windows XP Professional SP2.

Example 1. Synthetic data.

In Example 1, several data sets with size 5,000 and dimensions from 8 to 40 are generated. Seventy-two cluster centers are evenly distributed over the hypercube $[-1, 1]^d$ with d ranging from 8 to 40. A Gaussian distribution with standard deviation $\sigma = 0.05$ along each coordinate is used to generate data points around each center, where each coordinate is generated independently. Tables 1 and 2 list the computing time and number of distance calculations, respectively, for four algorithms to merge data points into 72 clusters.

From Tables 1 and 2, we may find that the PNN has the longest computation time and the largest number of distance calculations, while our method MFPNN_FS has the best performance in all cases. Compared to PNN algorithm, our method (MFPNN_FS) can effectively reduce the computing time and distance calculations by a factor of 1351–1670 and 9622–38405, respectively, for data dimensions ranging from 8 to 40. Compared to FPNN, MFPNN_FS can also reduce the computing time and distance calculations by a factor of 5.1–6.0 and 40.1–132.2, respectively. Compared to FPNN, MFPNN can reduce the computing time and distance calculations by 1.3%–7.2% and 11.3%–12%, respectively. It is noted that MFPNN_FS is more remarkable, if a data set with larger dimension is used.

The performance of MFPNN_FS is highly affected by the cluster separation degree [10]. The cluster separation degree of a data set generated in Example 1 is related to the standard deviation σ of Gaussian distribution. The higher standard deviation of a cluster will

Table 1
The computing time (seconds) for four algorithms to cluster data with dimensions ranging from 8 to 40 into 72 clusters

Dimension	PNN	FPNN	MFPNN	MFPNN_FS
8	574.56	1.77	1.70	0.34
12	699.77	2.33	2.30	0.42
16	823.45	2.91	2.83	0.50
20	942.25	3.44	3.30	0.59
24	1060.39	3.95	3.77	0.66
28	1185.16	4.56	4.30	0.80
32	1310.61	5.17	4.84	0.88
36	1437.02	5.74	5.34	1.06
40	1560.55	6.27	5.81	1.06

Table 2
The number of distance calculations for four algorithms to cluster data with dimensions ranging from 8 to 40 into 72 clusters

Dimension	PNN	FPNN	MFPNN	MFPNN_FS
8	20 833 270 304	71 692 760	63 602 015	542 454
12	20 833 270 304	76 309 913	67 510 671	911 516
16	20 833 270 304	79 492 604	70 239 288	1 052 705
20	20 833 270 304	81 817 553	72 040 936	1 219 972
24	20 833 270 304	82 654 690	72 741 113	1 286 211
28	20 833 270 304	84 579 954	74 523 930	1 548 205
32	20 833 270 304	86 139 152	75 936 525	1 719 229
36	20 833 270 304	86 902 658	76 644 849	2 165 049
40	20 833 270 304	87 312 045	76 797 728	2 110 969

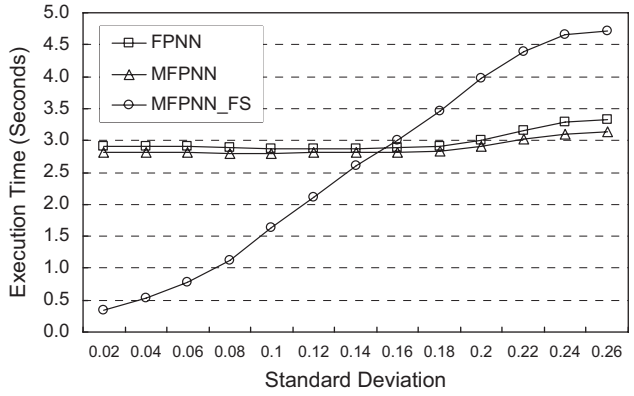

Fig. 1. The execution time for FPNN, MFPNN, and MFPNN_FS for data sets with dimension = 16 and various standard deviations.

Table 3
The computing time and distance calculations for four algorithms to generate a codebook of size 256

Methods	Computing time (s)	Distance calculations (times)
PNN	> 100 000	$> 1.0\times10^{12}$
FPNN	923.3	8 686 314 889
MFPNN	475.4	4 338 952 698
MFPNN_FS	37.2	59 346 789

cause a worse cluster separation. Fig. 1 gives the execution time for FPNN, MFPNN, and MFPNN_FS with dimension = 16 and various standard deviations.

From Fig. 1, we can see that the MFPNN is better than FPNN in all cases. Comparing to FPNN and MFPNN, MFPNN_FS has the less execution time for the case of standard deviation less than 0.16. The improvement of MFPNN_FS on execution time is more significant, if the standard deviation is smaller. That is, the MFPNN_FS is more suitable for a data set of high degree of cluster separation. It is noted that the real data set usually has the high degree of cluster separation and our method MFPNN_FS may have good performance for this kind of data set.

Example 2. Data set from real images.

In this example, a set of data points is obtained from three gray images: "Peppers," "Lena," and "Baboon." The image size of these images is 512×512. Each data point is an image block of 4×4 pixels and has the dimension of 16. That is, this data set contains 49 152 blocks (data points). Table 3 lists the computing time and the number of distance calculations for four algorithms to generate a codebook of size 256 using the data points from real images.

From Table 3, we can see that MFPNN_FS has the best performance in terms of the computing time and distance calculations. Compared to PNN, FPNN can reduce the computing time and distance calculations dramatically. However, our method MFPNN can further reduce the computing time and distance calculations of FPNN by a factor of 1.94 and 2.00. Compared with FPNN, our method MFPNN_FS can reduce the computing time and distance calculations significantly. MFPNN_FS can reduce the computing time and distance calculations of FPNN by a factor of 24.8 and 146.4, respectively. The good performance for applying MFPNN_FS on real images is due to real images having the high degree of cluster separation.

5. Conclusions

In this paper, we present a new method to improve the fast exact PNN algorithm using the property that the cluster distance increases as the merging process proceeds. Our proposed method also integrates a fast search algorithm to reject impossible clusters in the process of searching the nearest neighbor of a cluster. Experimental results show that our proposed method can effectively reduce the number of distance calculations and computing time of PNN and FPNN. Compared with FPNN, our proposed method MFPNN can reduce the computational time and number of distance calculations by a factor of 1.9 and 2.0, respectively, for the data set from three real images. Compared to FPNN, our proposed MFPNN_FS can reduce the computing time and number of distance calculations by a factor of 24.8 and 146.4, respectively, for the data set from three real images. It is noted that our method generates the same clustering results as those of PNN and FPNN.

Acknowledgement

This work was supported by a grand from National Science Council of Taiwan, ROC under grand no. NSC-96-2221-E-343-006.

References

[1] A. Gersho, R.M. Gray, Vector Quantization and Signal Compression, Kluwer Academic Publishers, Boston, MA, 1991.
[2] Y.C. Liaw, J.Z.C. Lai, Winston Lo, Image restoration of compressed image using classified vector quantization, Pattern Recognition 35 (2) (2002) 181–192.
[3] J. Foster, R.M. Gray, M.O. Dunham, Finite state vector quantization for waveform coding, IEEE Trans. Inf. Theory 31 (3) (1985) 348–359.
[4] S. Theodoridis, K. Koutroumbas, Pattern Recognition, second edition, Academic Press, New York, 2003.
[5] U.M. Fayyad, G. Piatetsky-Shapiro, P. Smyth, R. Uthurusamy, Advances in Knowledge Discovery and Data Mining, MIT Press, Boston, MA, 1996.
[6] D. Liu, F. Kubala, Online speaker clustering, Proc. IEEE Conf. on Acoust. Speech Signal Process. 1 (2004) 333–336.
[7] P. Hojen-Sorensen, N. de Freitas, T. Fog, On-line probabilistic classification with particle filters, Proc. IEEE Signal Process. Soc. Workshop 1 (2000) 386–395.
[8] M. Eirinaki, M. Vazirgiannis, Web mining for web personalization, ACM Trans. Internet Technol. 3 (2003) 1–27.
[9] Y. Linde, A. Buzo, R.M. Gray, An algorithm for vector quantizer design, IEEE Trans. Commun. 28 (1) (1980) 84–95.
[10] A.E. Cetin, V. Weerackody, Design vector quantizers using simulated annealing, IEEE Trans. Circuits Syst. 35 (12) (1988) pp. 1550.
[11] W.H. Equitz, A new vector quantization clustering algorithm, IEEE Trans. Acoust. Speech Signal Process. 37 (10) (1989) 1568–1575.
[12] J. Shanbehzadeh, P.O. Ogunbona, On the computational complexity of the LBG and PNN algorithm, IEEE Trans. Image Process. 6 (4) (1997) 614–616.
[13] T. Kurita, An efficient agglomerative clustering algorithm using a heap, Pattern Recognition 24 (3) (1991) 205–209.
[14] P. Fränti, T. Kaukoranta, D.F. Shen, K.S. Chang, Fast and memory efficient implementation of the exact PNN, IEEE Trans. Image Process. 9 (5) (2000) 773–777.
[15] T. Kaukoranta, P. Fränti, O. Nevalainen, Vector quantization by lazy pairwise nearest neighbor method, Opt. Eng. 38 (11) (1999) 1862–1868.
[16] J.Z.C. Lai, Y.C. Liaw, Fast searching algorithm for vector quantization using projection and triangular inequality, IEEE Trans. Image Process. 13 (2) (2004) 1554–1558.
[17] J. McNames, A fast nearest-neighbor algorithm based on a principal axis search tree, IEEE Trans. PAMI 23 (9) (2001) 964–976.
[18] J.S. Pan, Z.M. Lu, S.H. Sun, An efficient encoding algorithm for vector quantization based on subvector technique, IEEE Trans. Image Process. 12 (3) (2003) 265–270.
[19] Zhe-Ming Lu, Shu-Chuan Chu, Kuang-Chih Huang, Equal-average equal-variance equal-norm nearest neighbor codeword search algorithm based on ordered Hadamard transform, Int. J. Innovative Comput. Inf. Control 1 (1) (2005) 35–41.
[20] S.C. Tai, C.C. Lai, Y.C. Lin, Two fast nearest neighbor searching algorithms for image vector quantization, IEEE Trans. Commun. 44 (12) (1996) 1623–1628.
[21] T. Kanungo, D. Mount, N. Netanyahu, C. Piatko, R. Silverman, A. Wu, An efficient k-means clustering algorithm: analysis and implementation, IEEE Trans. PAMI 24 (7) (2002) 881–892.

About the Author—YI-CHING LIAW received his B.S., M.S., and Ph. D. degrees in Information Engineering and Computer Science from Feng-Chia University, Taiwan, in 1992, 1994, and 2004, respectively. From 1999 to 2004, he was an engineer with IndustrialTechnology Research Institute, Hsinchu, Taiwan. Since 2005, he has been working as an assistant professor at the Department of Computer Science and Information Engineering, Nanhua University, Chiayi, Taiwan. His current interests are in data clustering, fast algorithm, image processing, and video processing.